Fourier Neural Operators, with an Application to Source/Sink Fluid Simulation

This report has two parts. The first develops the idea of a Fourier Neural Operator (FNO) from the ground up, assuming no machine-learning background. The second documents a concrete application: a small FNO trained to advance a source/sink smoke field one step in time, served through a page where the user places sources and sinks and views the flow. The theory is self-contained; the application shows it in use, including where it fails. Some rigor is traded for clarity throughout.

---

1. Motivation

The goal is a fluid that responds to editing. The user clicks to place a source, a region that pushes smoke out, or a sink, a region that pulls it in, and the flow updates immediately, so a source can be moved and its effect seen at once. That requirement rules out waiting on a full solver.

The accurate way to produce the flow is a fluid solver, and a solver is slow in one specific respect. The box is cut into a grid, 64 cells across and 64 down. The solver stores a few numbers per cell (velocity in two directions, and smoke) and at each timestep enforces the rules of a fluid: smoke is carried along by the velocity, the velocity is driven by the sources and sinks, and a large coupled system is solved so the flow neither creates nor destroys volume. That final step ties every cell to every other at once. It is accurate and expensive. A 40-frame clip is 40 rounds of it, and moving a source discards all previous work and restarts.

That inefficiency is the opening. The solver has no memory: it computes the same kind of "field now to field one step later" mapping thousands of times and retains nothing. The alternative is to learn the mapping. Fit it once from many solved examples, then apply it at run time instead of re-deriving it. The solver runs once to generate training data; afterward each step costs milliseconds. This trade is the reason neural operators exist, and everything below serves it.

2. From networks to operators

A standard neural network takes a fixed input. A digit classifier maps 784 pixels to 10 scores; trained at 28×28, it cannot accept a 56×56 image. It is tied to one grid size.

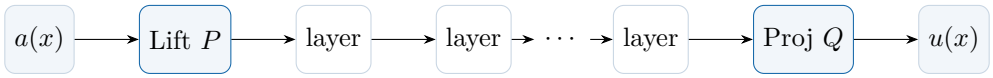
A fluid field is not fundamentally a grid. It is a smooth quantity defined over the box, a function of position, sampled at 64×64 for convenience. Sampled more finely, it is the same field with more measurements. The model should learn the relationship between the whole input field and the whole output field, so that a finer sampling still works. This property is called **discretization invariance**, and a standard network does not have it.



A network maps points to points and is fixed to one grid; an operator maps whole fields to whole fields. The rest of the theory is how to build a layer that acts on a field as a whole.

3. Structure of a neural operator

Every neural operator has the same three-part structure.



Lift expands the few input values at each point into a wider vector of channels, roughly 32 of them, giving each point capacity to carry information. **Project** reduces that vector back to the output values at each point. Both are small per-point linear maps. The work is done by the **layers** between them, four in this model, and each layer must perform the same difficult operation: mixing information across the whole box.

The mixing is necessary because physics is non-local. Smoke here a moment later depends on velocity elsewhere, and the pressure at a point depends on the whole field, which is the coupled step from Section 1. The standard expression for "every point y influences the new value at x, with a weight" is a kernel integral,

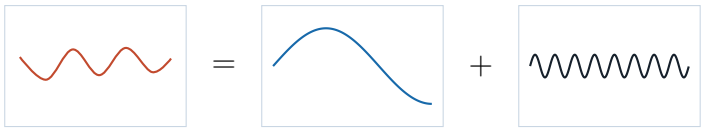
$$(\mathcal{K}v)(x) \;=\; \int \kappa(x,y)\, v(y)\, dy.$$

Here *kernel* means a weighting function, unrelated to the kernel of linear algebra: $\kappa(x,y)$ gives the influence of point y on the new value at x. In words, the new value at x is a weighted sum over every point y. This is a weighted average across the entire box.

Evaluated directly, it costs about N^2: each of N points summed against all N points, per layer, per step. On a 64×64 grid that is four thousand points against four thousand, far too slow for interactive use. The FNO reduces this cost, and the reduction relies on Fourier transforms.

4. Fields as a sum of waves

The method rests on one fact. Any field, however irregular, can be written as a sum of waves: a few large, slow, smooth waves for its broad shape, and many small, fast waves for fine detail and sharp edges. Low frequency corresponds to large and smooth; high frequency to fine and sharp.



a messy field | big smooth wave (keep) | tiny wiggles (often discarded)

The Fast Fourier Transform (FFT) returns how much of each wave a field contains, one coefficient per frequency; transforming back recovers the field. The fact that makes the method work is that smooth physics, such as soft smoke, is almost entirely composed of the large slow waves. The fast wiggles carry very little of it.

5. The Fourier layer

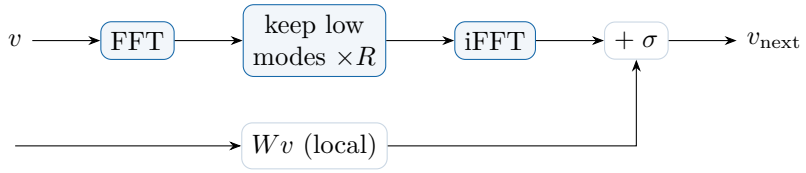
The convolution theorem states that the expensive weighted average of Section 3 becomes an ordinary multiplication in the frequency domain. Each FNO layer therefore replaces the N^2 integral with the following:

1. Transform the field to its wave coefficients. `x_ft = rfft2(x)`

2. Retain only the lowest wave numbers, 12 in this model, and discard the rest. Multiply each retained frequency's channel vector by a small learned complex matrix R. `out_ft[:modes] = R * x_ft[:modes]`
3. Transform back to ordinary space.
4. Add a cheap local term Wv, a per-point linear map, and apply a smooth nonlinearity σ (GELU, a smooth version of "keep positive values").

A full layer is

$$v_{\text{next}}(x) \;=\; \sigma\Big(\underbrace{Wv(x)}_{\text{local, preserves sharpness}} \;+\; \underbrace{\mathcal{F}^{-1}\big(R \cdot \mathcal{F}(v)\big)(x)}_{\text{global, smooth mixing}} \Big).$$



Two consequences follow. First, speed: the FFT costs about $N \log N$ rather than N^2. Second, resolution independence: R has an entry for each retained *wave number*, not for each grid cell, so it does not depend on how finely the field was sampled. The same trained R applies on a finer grid without change, allowing training on coarse data and evaluation on fine. This is the discretization invariance of Section 2. Discarding the fast wiggles is both the source of the speed and, as the next section notes, the source of the blur.

6. Limitations of the method

The choices that make the FNO fast also produce predictable failures.

It smears sharp features. Discarding high-frequency components is a blur. A crisp edge or a shock resides precisely in the discarded frequencies, so the FNO softens it. The method performs well on smooth flow and degrades as the field becomes sharp and multi-scale, which is one reason turbulence is difficult for it.

It assumes a periodic domain. The FFT treats the box as wrapping around, right edge joined to left. This matches the periodic fluid box used here, but irregular domains and solid walls require additional handling.

It is limited to its training distribution. The model learns a pattern from examples rather than the governing law. Given an input unlike its training data, it extrapolates from resemblance and is often wrong. It has learned the mapping, not the mechanism.

This concludes the theory. The remainder describes the application.

---

7. Input representation

The fluid occupies a 64×64 periodic box. Its state at frame t is three values per cell,

$$q_t(x, y) \;=\; \big[\, v_x,\; v_y,\; s \,\big](x, y, t),$$

the two velocity components and the smoke concentration s. This is the quantity the solver produces and the network advances.

The network also requires the positions of the sources and sinks, which are not present in q_t. The placement is encoded as an additional field, a signed **layout** $L(x, y)$: each source contributes a smooth positive region, each sink a smooth negative one. The layout is fixed over the clip. Stacked together, each input has six channels,

$$\big[\ \underbrace{v_x,\, v_y,\, s}_{\text{state}},\ \underbrace{L}_{\text{layout}},\ \underbrace{x,\, y}_{\text{coordinates}}\ \big] \longmapsto \big[\, v_x,\, v_y,\, s \,\big]_{\text{next frame}}.$$

Six channels in, three out. The two coordinate channels are included because the spectral layers operate in the frequency domain, which is insensitive to absolute position; supplying the raw x and y of each point lets the network distinguish locations.

One design limitation should be noted. The data generator samples a push *direction* for each source and stores it, but the network receives only the scalar layout, not the direction. This checkpoint therefore cannot be conditioned on push direction. Adding two force-component channels would address this, at the cost of regenerating the inputs and retraining. The current interface exposes placement, radius, strength, and source-or-sink type.

8. Network architecture

The model follows the lift/mix/project structure of Section 3, at a small size: width 32, 12 retained Fourier modes, 4 spectral layers, and a final projection through 128 channels to the 3 outputs. It has 1,188,707 trainable parameters.

One implementation detail is worth recording. The two-dimensional real FFT (`rfft2`) is one-sided in one axis and two-sided in the other, so it contains both the low positive and the low *negative* frequencies, with the negative frequencies wrapping to the far end of the array. Retaining the lowest modes therefore requires two blocks, not one: the top-left block for positive k_x and the bottom-left block for negative k_x. Each spectral layer holds two weight tensors accordingly. Retaining only one block silently discards half of the intended frequencies, with no error to indicate the loss.

9. Training

The dataset is 300 solver runs of 40 frames each, for 12,000 frames. The main risk is information leakage between training and test data. Splitting individual frames would allow frame 10 of a run to train the model while frame 11 of the same run evaluates it, and the two are nearly identical, so the model would appear strong while generalizing poorly. The split is therefore by whole run: 240 runs for training and 60 held out, with a fixed seed. This yields 9,360 training pairs and 2,340 test pairs with no leakage.

The model is scored by relative L2 error, the error magnitude divided by the magnitude of the true field, which places small and large fields on a comparable scale. Training is standard supervised learning: Adam at learning rate 10^{-3}, light weight decay, the rate halved every 50 epochs, over 200 epochs, with the best test checkpoint retained. The governing equations do not appear in the loss; the model observes only input-output pairs produced by the solver. It replaces re-solving the physics, not knowing it.

10. Single-step accuracy and rollout instability

The model predicts a single step, from the current state to the next. Producing a full clip requires running it in a loop, feeding each prediction back as the next input, an autoregressive rollout. This introduces a failure mode: each prediction carries a small error, and feeding a slightly incorrect frame back in means the following prediction begins from an imperfect input, so errors accumulate.

Measured one step at a time, accuracy is high: low test error, sharp predicted frames, and a stable training curve. Run autoregressively, the first frame is accurate, the second slightly degraded, and within a few frames the field collapses to a nearly uniform region. No error is raised, which makes the failure easy to overlook.

The cause is a distribution shift. During training, the model always predicts from a real solver frame, which is clean and representative. At run time it predicts from its own previous output. Even an accurate prediction lies slightly outside the set of states the solver produces, and once that output is fed back, the input is off-distribution and the error compounds until the rollout diverges.

The demonstration avoids this by keeping the solver in the loop. For each frame, the solver computes the true next state and the FNO predicts one step forward from it, so every displayed frame is produced from an in-distribution input. The cost is that the solver runs on every request, making the deployment a guided demonstration rather than a standalone surrogate. Removing this dependency is the first item in the next section.

11. Future work

1. **Eliminate rollout drift.** Train on the model's own imperfect outputs, using a multi-step or pushforward loss, so it remains stable without the solver in the loop. This is the highest-value change and restores the original objective.
2. **Condition on direction.** Add the two force-component channels so the network can respond to the push direction of each source. Requires regenerating inputs and retraining.
3. **Consolidate the checkpoint.** Weights, architecture, normalization statistics, and the split seed should be stored as a single artifact rather than in separate files.
4. **Broaden the data.** 300 runs is limited coverage, particularly near the periodic boundary and at extremes of forcing strength.
5. **Increase spectral resolution as needed.** Raise the mode count or adopt a multi-resolution architecture when blur becomes limiting.
6. **Calibrate physical time.** Define the solver's seconds-per-step; the "5 seconds" in the demo is a display rate (8 frames per second), not calibrated physical time.

12. Summary

A solver computes each answer independently and from scratch. A Fourier neural operator instead learns the map from the current field to the next from examples the solver generated once, and performs the global mixing step efficiently by transforming to the frequency domain, retaining the low-frequency modes, multiplying by learned weights, and transforming back, which also makes it resolution-independent. The model presented here is trained on a source/sink smoke field. Single-step predictions are accurate, while autoregressive rollouts drift, because a one-step model applied to its own output leaves the training distribution; the deployed demonstration therefore keeps the solver in the loop and advances one step at a time. The method is suited to smooth problems solved repeatedly and blurs sharp structure.

References

[1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar. Fourier Neural Operator for Parametric Partial Differential Equations. ICLR 2021. arXiv:2010.08895.

[2] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar. Neural Operator: Learning Maps Between Function Spaces. 2023. arXiv:2108.08481.

[3] PhiFlow. https://tum-pbs.github.io/PhiFlow/.

[4] Project source: `model.py`, `generate_data.py`, `train.py`, `evaluate_rollout.py`, `app.py`.